PROJECT AND TEAM INFORMATION

Project Title

GRAPH BASED FRIEND AND OPPORTUNITY RECOMMENDATION SYSTEM

Student / Team Information

Team Name: *Team #*	*Tech-mates* *DSCPP-III-2026-T209*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Aaradhya – 2510010209* *2510010209@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Rana ,Deepak Singh – 2510380196* *2510380196@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Nagrey,Vanshika – 2510370219* *2510370219@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Raju ,Akshra – 25022036* *25022036@geu.ac.in* 

PROPOSAL DESCRIPTION

Motivation

With the increasing number of people and connections on social and professional platforms,finding relevant people and meaningful opportunities can become difficult .Traditional social networking systems mainly focus on connecting users based on existing friendships,while useful similarities such as common interests,skills, courses,workshops,projects,seminars,competitions and other experiences may not always be effectively utilized . For example,two students may not know each other but may have several common interests,skills,courses or projects .Similarly,a student may benefit from being connected with someone who has participated in a particular workshop,competition or seminar. Our project aims to address this problem by developing a Friend Opportunity Recommendation System that represents relationships using a graph and recommends users or learning opportunities based on their connections and common attributes . The project is also motivated by the need to demonstrate how Data Structures and Algorithms can solve a real world problem .Graph traversal algorithms such as BFS and DFS can be used to explore social connections, while hashing and priority queues can improve searching and recommendation ranking .

State of the Art / Current solution

Existing social-network recommendation approaches commonly use the concept of friends-of-friends. A user can be represented as a node in a graph and a friendship as an edge. Users who are connected through mutual friends can then be considered potential recommendations. A simple example is:
D-B-A
D-C-A
Here, A and D are not directly connected, but they have common connections. Such relationships can be identified using graph traversal. Our proposed system extends this idea by considering not only mutual friends, but also: Common interests,Skills,Courses,Seminars,Workshops,Competitions/Hackathons,Certifications,Projects,Webinars. The system represents friendships using an undirected graph with an adjacency list implemented using linked lists . BFS can explore users according to their distance from a particular user, while DFS can explore deeper connections. Common attributes are compared to calculate a recommendation or compatibility score.The generated recommendations can then be ranked using a priority queue. Neo4j is used as the graph database and persistent storage layer. It stores users, friendships, interests and activities. When required, the data can be loaded into the C++ data structures for graph processing.

Project Goals and Milestones

The major goals of the project are:
Develop a C++-based friend and opportunity recommendation system. Represent social relationships using an undirected graph . Implement the graph using an adjacency list with linked lists . Implement BFS and DFS for graph traversal. Provide fast user searching using hashing. Identify mutual friends and common interests . Calculate a compatibility/recommendation score . Rank recommendations using a priority queue. Provide learning and experience recommendations. Store and retrieve information using Neo4j . Apply OOP concepts to organize the complete system. Initial Milestones
Milestone 1 *— Requirement Analysis Identify system features. Define user information and relationship types . Decide the required DSA concepts.*
Milestone 2 *— C++ and OOP Structure Create classes such as User,Activity,Graph,etc . Define functions for user and relationship management .*
Milestone 3 *— Graph Implementation Create an undirected graph. Implement adjacency lists using linked lists. Implement adding and removing connections.*
Milestone 4*— Graph Algorithms Implement BFS. Implement DFS. Implement mutual-friend detection. Explore users based on connection distance.*

Project Approach

We are tackling this with a modular C++ design,keeping each component focused on a specific job .User management is straightforward-classes hold IDs,skills,courses,and certifications,with basic operations to add,remove,or search .

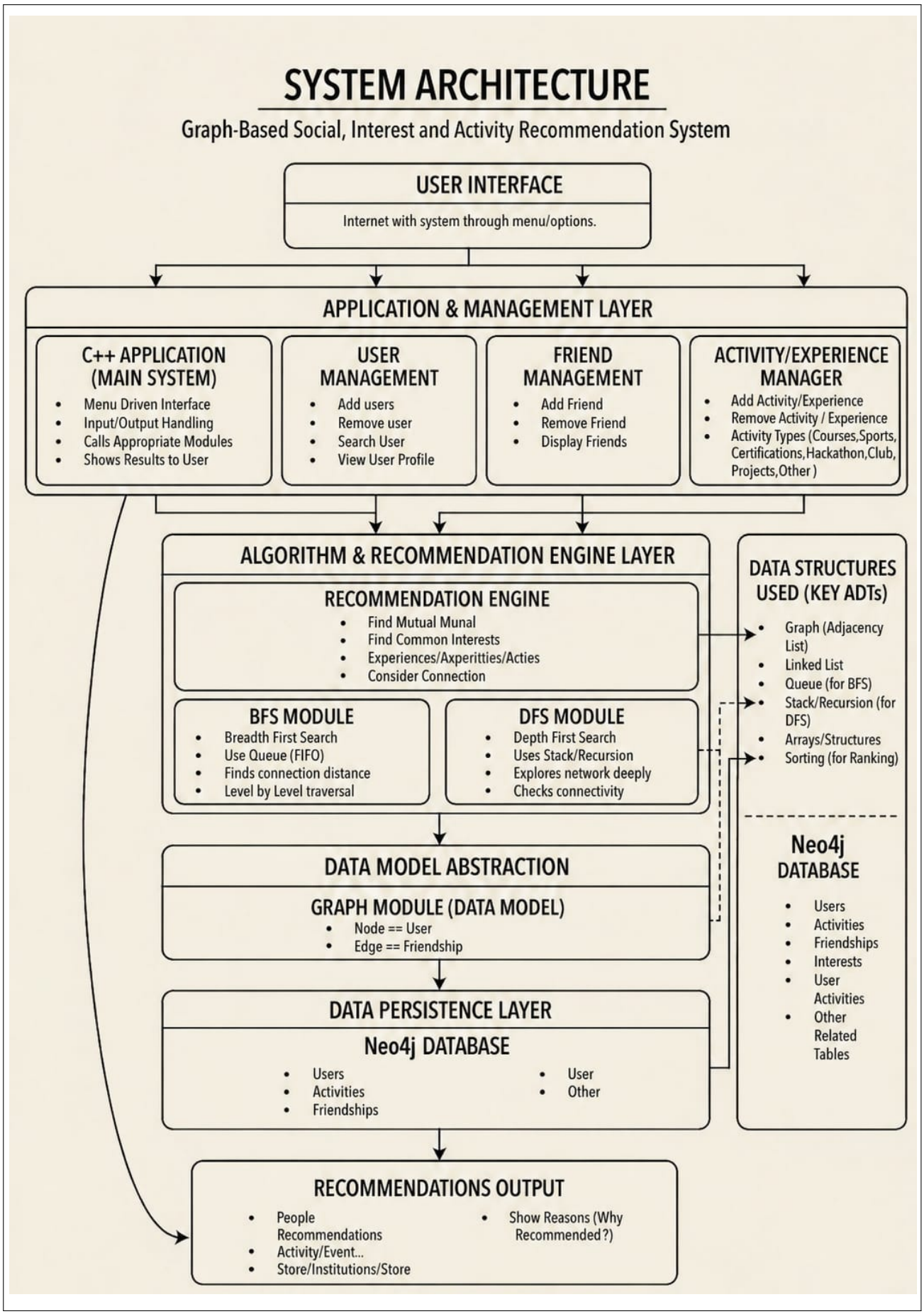
For the social layer,we map users to an undirected graph .Users are vertices,friendships are edges .But instead of using a standard vector for adjacency,we are implementing the list using custom linked lists .Why ?Because this project is as much about demonstrating DSA fundamentals as it is about recommendations,and we want to show we can manage dynamic memory and traversal manually .

For exploration,BFS is our workhorse for distance-based discovery-level 0 is the user,level 1 is direct friends,level 2 is friends-of-friends .DFS complements this by digging deep into specific chains,which is useful for checking connectivity or tracing particular relationship paths .

Ranking candidates depends on a weighted score .We factor in mutual friends and overlapping profile attributes-common skills,projects,workshops,or seminars. We will settle exact weights during implementation .To keep lookups fast,we use unordered map hashing for attributes .Once we compute scores,candidates go into a priority queue Aman with a 9 gets popped before Rahul with a 5,giving us a clean ranked list .

Neo4j handles persistent storage-users,friendships,and activities live there .But we are strict about the split the database only dumps raw records .The processing happens entirely in C++ .The flow is simple load records from Neo4j,instantiate our C++ objects,build the linked-list graph,run BFS/DFS for exploration,calculate scores,push to the priority queue,and output the final recommendations .This keeps storage and computation separate,making the system easier to maintain and faster to execute .

System Architecture (High Level Diagram)



Project Outcome / Deliverables

We'll end up with a working system that handles the full cycle: managing users and friendships,exploring the network through BFS and DFS and surfacing ranked recommendations based on shared skills, interests and mutual connections. On the storage side Neo4j will persist everything-users,relationships,activities, so the data doesn't get lost between runs .
Beyond the functional features,the real outcome is a tangible demonstration of DSA in action .We're building custom linked lists for graph adjacency,using hashing for fast profile lookups,leveraging priority queues for ranking and tying it all together with clean OOP design in C++ .It's not just a recommendation tool,its a proof point that these core data structures and algorithms can handle a messy real-world problem like finding relevant people or opportunities in a crowded network.

Assumptions

The following assumptions are made for the development of the system:
-Every user has a unique User ID. A friendship is considered mutual, therefore the friendship graph is undirected.
-Users provide their interests, skills and activities as profile information.
-The recommendation score is based only on the information available in the system.
-A higher compatibility score indicates a stronger recommendation.
-The system does not guarantee that a recommended person will actually become a friend.
-Neo4j is used for local/persistent storage rather than as the graph-processing engine.
-The graph is constructed in C++ from the stored relationship data.
-The initial system is intended as an academic prototype and may not handle the scale of a large commercial social-media platform.
-The recommendation algorithm is rule-based, rather than using machine learning.

References

1. Research paper / journal -
https://www.scribd.com/document/968694521/Social-Network-Friend-Recommendation
https://www.scribd.com/document/968694521/Social-Network-Friend-Recommendation
https://recsysml.substack.com/p/friend-recommendation-retrieval-in

2. Book / standard reference on Data Structures Algorithms

3. Official C++ / STL / Neo4j documentation

4. Dataset / repository